2609 CS460

Computer Vision

Homework3: Stereo Vision

Name: 李昌硕

Student ID: 1240000787

Task 1: Estimating the projection matrix M

Task 1 (a) find_projection

Each correspondence satisfies $p \equiv MX$, with $p = [u, v, 1]^T$ and $X = [x, y, z, 1]^T$. The cross product $p \times MX = 0$ removes the unknown scale. Only two of the three resulting equations are independent. Writing m for the 12 entries of M stacked row by row, correspondence i contributes the two rows

```
[  0          X_i     -v_i X_i ]
[ X_i         0       -u_i X_i ]
```

Stacking every correspondence produces a matrix A with 2N rows. The fit is the homogeneous least-squares problem $Am = 0$. The solution is the right singular vector of A belonging to the smallest singular value, reshaped into a 3×4 matrix. That vector is returned at unit length, so the reported M has Frobenius norm 1. Any other scale represents the same camera.

```
X = [pts3d | 1]
A[0::2] = [0,  X, -v X]
A[1::2] = [X,  0, -u X]
M = svd(A).Vt[-1].reshape(3, 4)
```

Task 1 (b) M for task1/pts2d.txt and pts3d.txt

There are 20 correspondences. The unit-norm projection matrix is

```
[ 0.45827554  -0.29474237  -0.01395746   0.00402580]
[-0.05085589  -0.05458470  -0.54105993  -0.05237592]
[ 0.10900958   0.17834548  -0.04426782   0.59682050]
```

The scale is arbitrary: if M is a solution then so is λM, because the homogeneous image coordinate is divided by its third entry. The matrix above is the one with $\|M\|_F = 1$ returned by the SVD.

Task 1 (c) Average reprojection distance

`compute_distance` projects every 3D point with this M, converts the result from homogeneous to pixel coordinates by proj([x, y, w]) = [x/w, y/w], and returns the mean Euclidean distance to the given 2D points. On the task1 sample that distance is 0.002227 pixels, below the 0.01 threshold in the assignment. The points are therefore consistent with a single linear camera.

Task 1 (d) Equation 2 and Equation 6

The two expressions are related, but they are not the same objective.

Equation 6 is the matrix form of $p \times MX = 0$. It is linear in the entries of M, which is why it can be solved by homogeneous least squares. Its residual on one correspondence is made of terms $M_1 \cdot X - u\, M_3 \cdot X$ and $M_2 \cdot X - v\, M_3 \cdot X$. If $w = M_3 \cdot X$ is the homogeneous scale of the projected point, those terms equal w times the pixel error in x and in y. The quantity that the SVD actually minimizes is the sum of squares of these algebraic residuals.

Equation 2 is the mean Euclidean distance, in pixels, between the observed point and the dehomogenized projection. It is the geometric reprojection error. It equals the algebraic residual only after dividing by w. Because w depends on depth and on the arbitrary scale of M, the algebraic objective weights points differently from the geometric one.

On this sample there is no measurement noise, so both residuals can be driven essentially to zero (the geometric mean is 0.002227 px). The two objectives then pick out the same camera. With noisy measurements they do not. Equation 2 is the one that should be minimized: it is an actual distance in the image, in pixel units, and it does not depend on how M is scaled. Equation 6 does not measure a geometric distance. It is still the right tool for a first estimate, because it is linear and has a closed-form solution. A noisy problem would use that estimate to initialize a nonlinear minimization of Equation 2.

Task 2: Estimating the fundamental matrix

Task 2 (a) Eight-point algorithm

`find_fundamental_matrix` follows the normalized eight-point algorithm.

1. Each image gets its own similarity T. The point set is translated so that its centroid is at the origin, then scaled so that the mean distance from the origin is √2. This is the normalization from Hartley's eight-point algorithm. It is the stable form of the instruction to center the data at 0 and to scale coordinates that otherwise range over hundreds of pixels. The image diagonal, $\sqrt{(h^2+w^2)}$ from the given image shape, is the fallback scale if the points are degenerate. Raw pixel products u′u would otherwise dominate the constant term 1 and make U^TU ill-conditioned.

2. For the normalized points, each correspondence supplies one row [u′u, u′v, u′, v′u, v′v, v′, u, v, 1], the Kronecker product of [u′, v′, 1] and [u, v, 1]. The vectorized F is the smallest right singular vector of this design matrix.

3. A fundamental matrix must have rank 2, otherwise the epipoles do not exist and the epipolar lines do not meet. The SVD of the fitted matrix is recomputed, the smallest singular value is set to 0, and the matrix is reconstructed. That is the closest rank-2 matrix in the Frobenius norm.

4. The normalization is undone by $F = T_2^T F_{rank2} T_1$. Substituting the normalized points $\bar{p} = T p$ into $\bar{p}'^T F_{rank2} \bar{p} = 0$ gives exactly $p'^T (T_2^T F_{rank2} T_1) p = 0$.

F is defined only up to scale. After dividing by the Frobenius norm and fixing the sign, the result agrees with cv2.findFundamentalMat(..., FM_8POINT), which uses the same normalization:

Dataset	Mean symmetric epipolar distance (px)	Matches OpenCV 8-point
temple	0.3592	1.74e-14
reallyInwards	0.3798	1.12e-16
xtrans	3.14e-13	7.94e-11

The residual of about 0.36 px on temple and reallyInwards is the same residual OpenCV reports. The correspondences are stored as integers, so a perfect real-valued F cannot put every point exactly on its line. On xtrans the geometry is a pure horizontal translation and the residual is about 10^{-13} px.

Task 2 (b) Epipoles

The epipole in image 1 lies on every epipolar line of image 2, so $F e_1 = 0$. The epipole in image 2 satisfies $e_2^T F = 0$. `compute_epipoles` reads these from the SVD $F = U\Sigma V^T$: e_1 is the last column of V, and e_2 is the last column of U. Both are returned as homogeneous coordinates, without forcing the third entry to 1. A third entry of 0 is a real answer: the epipole is then a point at infinity, and dividing by it is not defined. For every dataset, $\|F e_1\|$ and $\|e_2^T F\|$ are on the order of 10^{-16} to 10^{-19}.

Task 2 (c) Epipolar lines

The figures below are drawn by the supplied draw_epipolar routine, using every tenth correspondence so that the lines stay readable. A line and its matching points share a color. The epipole is drawn when it falls inside the image.

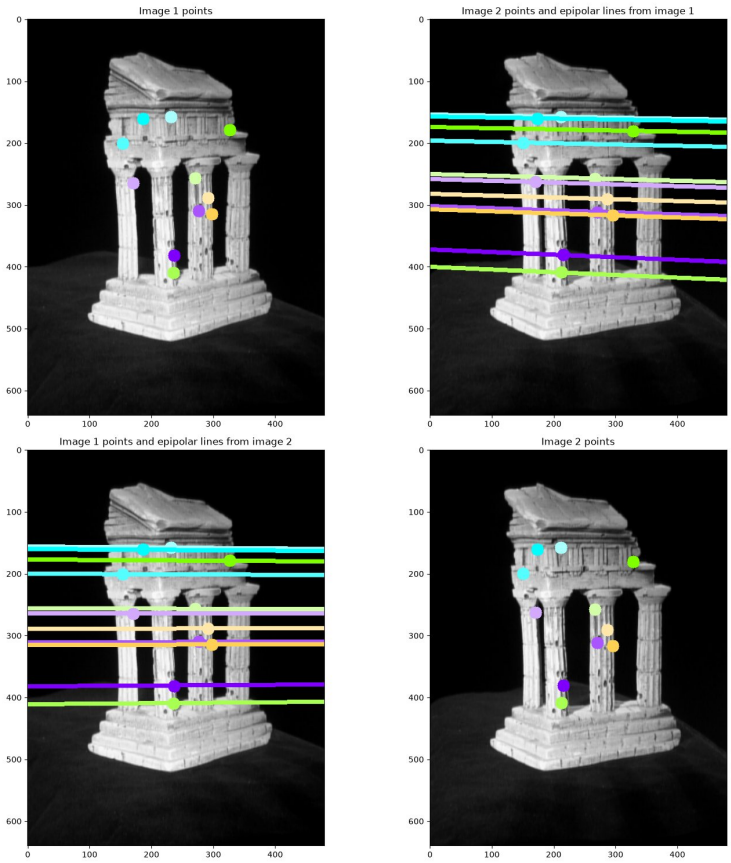


Figure 1. Temple. The epipoles are far outside the frame, at about (15206, 279) and (−9279, −2), so inside this narrow image the epipolar lines are nearly parallel. Each line passes through the point of the same color.

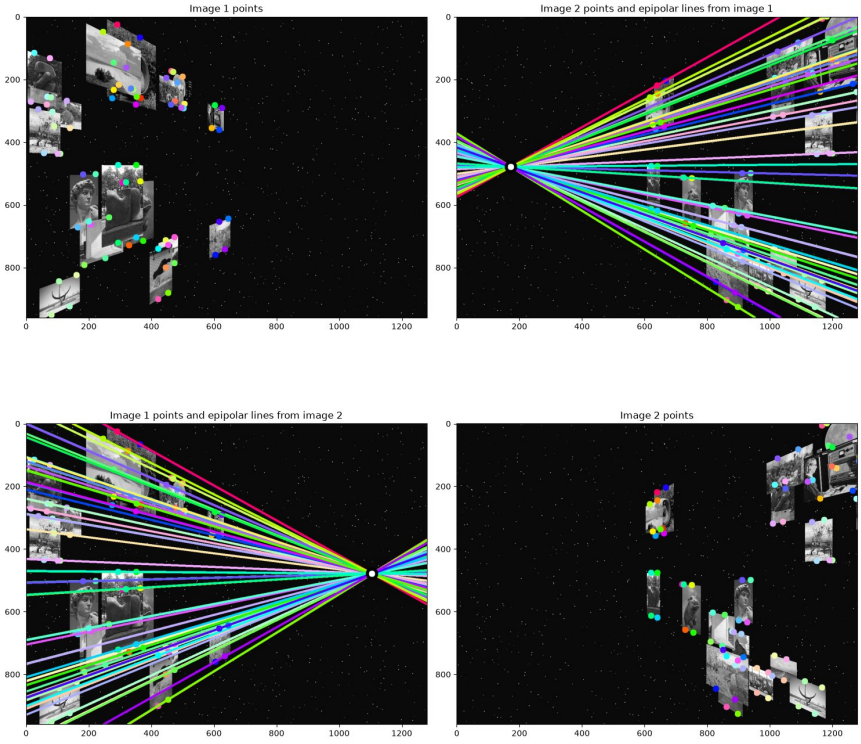


Figure 2. reallyInwards. The cameras are toed in, so both epipoles fall inside the images (the black-and-white dots). Every epipolar line passes through the epipole and through the matching point.

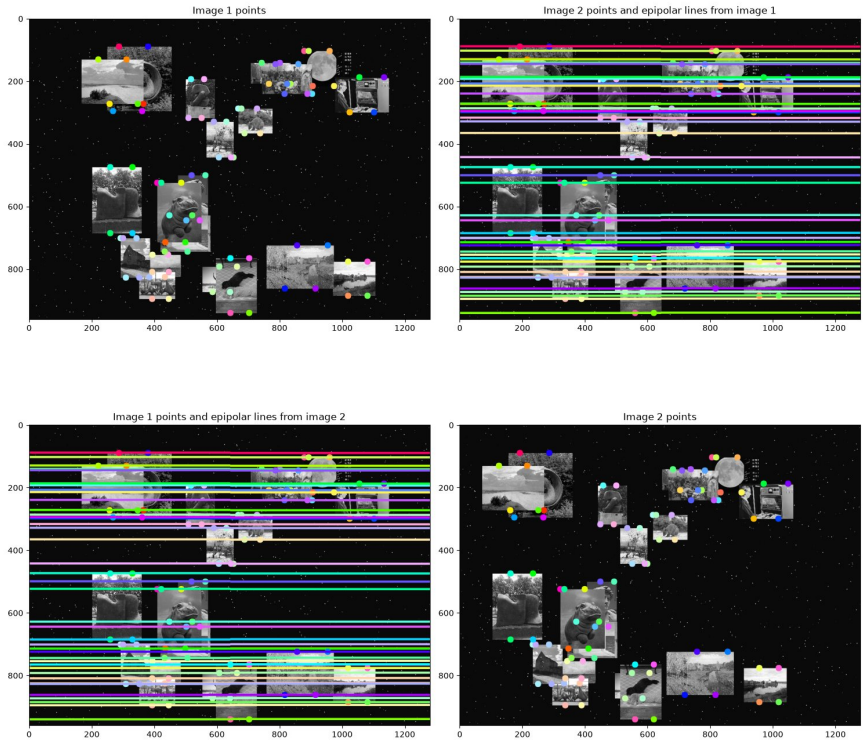


Figure 3. xtrans, the extra dataset. The motion is a horizontal translation, so the epipolar lines are horizontal and the epipoles are at infinity. No epipole is drawn, and points of one color stay on a common horizontal line.

Task 2 (d) Epipoles for reallyInwards and xtrans

Dataset	Epipole	Homogeneous (unit vector)	Image coordinates
reallyInwards	e1, image 1	[-0.917537, -0.39765, -0.000829639]	(1105.95, 479.31)
reallyInwards	e2, image 2	[0.339691, 0.940535, 0.00196184]	(173.15, 479.41)
xtrans	e1, image 1	[1, 3.42367e-15, 5.54217e-18]	at infinity
xtrans	e2, image 2	[-1, -3.0748e-15, 0]	at infinity

For reallyInwards the image is 1280×960 and the principal point is (640, 480). e_1 dehomogenizes to (1105.95, 479.31) and e_2 to (173.15, 479.41). Both are inside the image, at nearly the same height as the principal point, one on the right and one on the left. That is the pattern in Figure 2.

For xtrans, e_1 is [1, 0, 0] up to 10^{-15}, and the third entry of e_2 is exactly 0, so e_2 = [−1, 0, 0]. Both are points at infinity in the horizontal direction. Dividing by the third entry is undefined for e_2 and gives a meaningless coordinate near 10^{17} for e_1. The homogeneous vectors are the right quantities to report. They are why the lines in Figure 3 are parallel and horizontal: a pure sideways translation sends the baseline parallel to the image plane.

The same null-vector procedure gives the temple epipoles used in the caption of Figure 1, and it places the ztrans epipoles on the principal point, as a forward translation should.